Applying AI to Gentrification

Analise Bottinger, Caitlin Flynn, Tyler Mckenzie

April 2025

Abstract

Gentrification can be defined as a process in which there are demographic changes, in particular where more affluent people move to a neighborhood and drive prices up which in turn drives existing residents out. Our paper has two parts, quantitative and qualitative. In the quantitative we used machine learning models to predict gentrification with moderate success. The qualitative part analyzes the relationship between public sentiment and neighborhood gentrification using Twitter data. The study examines how predicted sentiment distributions vary across neighborhoods with different gentrification statuses. Although the observed correlation between sentiment and gentrification is modest, the results suggest a potential connection that may be better understood with expanded temporal data and more comprehensive coverage.

1 Introduction

Our paper aims to explore the ability to predict gentrification status using various housing and demographic data points, some of which are not inherently obvious to be markers that a neighborhood is at risk for becoming gentrified. On the quantitative side we label the 55 sub-boroughs in NYC as high income (which implies it cannot be gentrified), non-gentrifying, and gentrifying. Often gentrification is a topic of political debate, we explore gentrification through a qualitative lens by sentiment analysis for various NYC neighborhoods and how this correlates to their respective gentrification statuses. The two of these combined allow us to better understand the process of Gentrification and how artificial intelligence methods can be used in understanding its impacts in neighborhoods.[1]

2 Background

In the following sections, we explain the algorithms used in our analysis:

[1] `github.com/anabottinger03/gentrification_project`

2.1 Feedforward Neural Network

The neural network implemented was feedforward, and implemented using Python and NumPy. The FNN works by processing the input features through the hidden layers, using ReLU activation function, and produces a probability distribution over the labels. A cross-entropy loss function was used to measure the difference from the softmax output and true labels. This loss is then used in the backpropagation process where we update each weight according to how much it contributed using gradient descent. Many experiments were run with this including varying learning rate, weighting to account for unbalanced data, number of epochs, number of neurons, number of hidden layers, size of batches, and drop out rate.

2.2 Decision Trees

Decision trees are supervised learning algorithms used for classification and regression tasks. They model decisions and their possible consequences in a tree-like structure, making them intuitive and interpretable. This process is recursive, creating branches until a stopping criterion is met (examples: maximum depth or minimum number of samples per leaf). Each node represents a feature, each branch represents a decision rule, and each leaf node represents an outcome. Our decision rule was based on entropy, each split tried to maximize the amount of information gain

2.2.1 Previous Applications

Alejandro and Palafox (2019) utilized classification trees to develop an index indicating the likelihood of a neighborhood undergoing gentrification. This approach demonstrated the utility of decision trees in capturing complex socio-economic patterns

2.2.2 Gradient boosting

To address the overfitting and instability issues of single decision trees, ensemble methods like Gradient Boosting are employed Gradient Boosting builds a bunch of small decision trees one after the other, where each new tree tries to fix the mistakes the last one made by minimizing loss.

2.3 Sentiment models

2.3.1 Logistic Regression

Logistic regression is an algorithm with the ability to handle multiple different explanatory variables, however, the output variable is binomial. This makes the algorithm adept at binary classification with multiple different influential input features. The foundation of the algorithm used in the paper can be defined by the following logit function and sigmoid function, representing the linear combination of each of the input features:

$$Logit(P(x)) = w_1x_1 + w_2x_2 + w_3x_3$$
$$Logit(P(y = 1|x)) = 1e(w^tx).$$

2.3.2 Naive Bayes

Naive Bayes is a supervised learning algorithm where each input feature is assumed to have conditional independence when determining the output class. This "Naive" assumption is applied to Bayes theorem to produce the classification output.

$$P(\theta|D) = \frac{P(\theta)}{P(D|\theta)}$$

2.3.3 Support Vector Machine (SVM)

SVMs are a type of supervised machine learning algorithm that utilize an optimal line to maximize the distance between two classes within a dataset. The base formula is as follows:

$$\begin{aligned} \min_{\mathbf{w},b} \quad & \frac{1}{2}\|\mathbf{w}\|^2 \\ \text{subject to} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \quad \text{for all } i \end{aligned}$$

3 Related Work

Some studies have employed CNNs to analyze visual data, such as Google Street View images, to detect physical signs of gentrification like building upgrades. For example, a study by Thackway et al. (2023) uses a Siamese CNN model to identify property upgrades as indicators of gentrification. However, the implementation of these models requires a lot of visual data, which may not be feasible to collect for all neighborhoods, and would require manual labeling, which would not be possible to do effectively in a limited time frame. Other studies have also employed similar techniques to attempt to explore the relationship between online sentiment and gentrification. In Hu, et. al. (2019), for example, they utilized a similar strategy in which they explore tweet data in order to quantify public perception towards gentrification. They used a Vader sentiment model combined with linear regression to determine if sentiment is a predictor of neighborhood improvement metrics. While the general method is feasible for our analysis, we were unable to replicate it at the same scale given the cost limitations of obtaining access to the full size X API.

4 Project Description

4.1 Data

4.1.1 Classification Data

For gentrification machine learning prediction, we gathered data from CoreData.nyc which is put together by the NYU Furman Center. The website contains multiple data points on New York City's neighborhoods, including demographics, development, housing, neighborhood services and conditions, and renters and rental conditions. There are 55 sub-borough areas, making for 55 rows. For each sub-borough we selected our data points of interest based on the paper "Identifying Gentrification using Machine Learning" by Jayne Yoo at the US Census Bureau. In this paper, a similar task of predicting gentrification in the Washington D.C metro area, the author lays out the features that were successful in building out their models. We used this to guide our features, while not following a one to one mapping, we selected similar features including education attainment, income diversity, racial diversity, crowding rate, age breakdown, home ownership rate, born in New York rate, foreclosure rate, proximity to parks, proximity to subway stops, elementary math and ELA proficiency.

The data we collected is unlabeled. To label our data for training, we used a NYU Furman Center paper, "State of New York City's Housing and Neighborhoods in 2015", where they outlined cutoffs for percentage changes in median income as their method to determine if a sub-borough is 1) Gentrifying 2) Non-Gentrifying 3) Higher-Income. We took this cut off point and directly integrated using change in median income in 2017 to 2019 our data to label each neighborhood as one of these 3 labels.

4.1.2 Sentiment Data

The Sentiment140 dataset is a collection of over 1.6 million sentiment-labeled tweets for classification-based tasks. The columns relevant to our training process are highlighted below:

- *Text*: the raw tweet text.
- *Target*: the polarity of the tweet (0 = negative, 4 = positive)

New York City neighborhood-specific tweets were collected using Twikit Twitter API scraper. This API scraper was chosen for no other reason other

than budget limitations preventing us from purchasing the X API. Due to Twikit's rate limit of 500 tweets per every 15 minutes, a custom POC script was used for scraping neighborhood specific tweets utilizing a search with backoff method, combined with Twikit's built in *searchtweet* method. The search tweet method is limited to only string queries and three filters: ('Top', 'Latest', 'Media'). 'Latest', or the most recently tweeted tweets containing the string search query was the chosen filter across all tweet collections. All in all, tweets were collected across 9 neighborhoods also found in our gentrification dataset (Table 1). with three representatives from each of our Gentrifying, higher income, and non-gentrifying categories. Due to time limitations, only an average of 170 tweets were collected per neighborhood for a total of 1553. In addition, Twikit does not allow for geolocation, so to ensure the highest probability that each of our collected tweets was indeed representative of said neighborhood, the borough in which the neighborhood is located was also added to the string search query. The scraped tweets were stored in CSV format with columns "neighborhood" and "tweet".

Neighborhood	Count
Sunset Park, Brooklyn	193
Flatbush, Brooklyn	189
Bushwick, Brooklyn	183
Astoria, Queens	175
Borough Park, Brooklyn	174
Central Harlem, Manhattan	169
Rockaways, Queens	168
Coney Island, Brooklyn	167
Upper East Side, Manhattan	135

Table 1: Counts by Neighborhood

4.2 Methods

4.2.1 Neural Network Algorithm

Algorithm 1 Train Feedforward Neural Network

Input: Training data $X_{\text{train}}, y_{\text{train}}$, test data $X_{\text{test}}, y_{\text{test}}$, learning rates $\eta \in \{0.001, 0.005, 0.01, 0.05\}$, batch size $B = 8$, epochs $E = 1000$, dropout rate $p = 0.3$
Output: Test accuracy for each η
for each η **do**
  Initialize weights and biases
  for $e = 1$ to E **do**
    Shuffle $X_{\text{train}}, y_{\text{train}}$
    for each batch (X_b, y_b) **do**
      for each $(x, y) \in (X_b, y_b)$ **do**
        Forward pass with ReLU, dropout, softmax
        Compute loss: $L = -\sum_c y_c \log(\hat{y}_c)$
        Backpropagate to update gradients
      end for
      Update weights using gradients and η
    end for
  end for
  Evaluate on X_{test}, compute accuracy
end for
Output results

4.2.2 Decision Tree/GBH Algorithm Equations

Algorithm 2 Train Decision Tree (Entropy-Based)

Input: Dataset D, maximum depth $d_{\max}$, minimum samples per leaf $s_{\min}$
Output: Decision Tree
function TrainTree(D, depth)
  if depth $\geq d_{\max}$ or number of samples in $D < s_{\min}$ **then**
    return Leaf node with majority class
  end if
  for each feature f **do**
    for each split point s in f **do**
      Split D into D_{left}, D_{right}
      Compute entropy gain of the split
    end for
  end for
  Choose f^*, s^* that gives max entropy gain
  Create node with split rule $f^* \leq s^*$
  Left child = TrainTree(D_{left}, depth+1)
  Right child = TrainTree(D_{right}, depth+1)
  return Node with children
end function

Algorithm 3 Gradient Boosted Decision Trees

Input: Training data X, y, number of trees T, learning rate η
Output: Ensemble of trees
Initialize model $F_0(x) = \arg\min_c \sum L(y_i, c)$
for $t = 1$ to T **do**
  Compute pseudo-residuals: $r_i = -\frac{\partial L(y_i, F_{t-1}(x_i))}{\partial F_{t-1}(x_i)}$
  Fit regression tree $h_t(x)$ to residuals r_i
  Compute multiplier γ_t that minimizes $\sum L(y_i, F_{t-1}(x_i) + \gamma_t h_t(x_i))$
  Update model: $F_t(x) = F_{t-1}(x) + \eta\gamma_t h_t(x)$
end for
return Final model $F_T(x)$

4.2.3 Sentiment Analysis Algorithm Equations

Algorithm 4 Train Logistic Regression Classifier

Input: $X_{\text{train}}, y_{\text{train}}, X_{\text{test}}, y_{\text{test}}$, learning rates $\eta \in \{0.01\}$, regularization types $\in$ {L1, L2, Elastic Net, None}, $\lambda = 0.2$, iterations $N = 1000$
Output: Final loss and weights θ for each (η, regularization)
for each η **do**
  for each regularization type **do**
    Initialize $\theta \leftarrow 0$
    for $i = 1$ to N **do**
      $z \leftarrow X_{\text{train}} \cdot \theta, \quad h \leftarrow \frac{1}{1+e^{-z}}$
      $\nabla \leftarrow X_{\text{train}}^{\top}(h - y_{\text{train}})$
      Apply regularization to ∇ based on type
      $\theta \leftarrow \theta - \eta \cdot \nabla$
      if $i \bmod 100 = 0$ or $i = N$ **then**
        Compute loss and check convergence
        if converged **then break**
        end if
      end if
    end for
    Save loss plot
  end for
end for
Output final θ and losses

Algorithm 5 Train Support Vector Machine

Input: $X_{\text{train}}, y_{\text{train}}$, learning rates $\eta \in \{0.01\}$, regularization types $\in$ {L1, L2, Elastic Net, None}, regularization strength $\lambda = 0.001$, iterations $N = 100000$
Output: Final weights w and hinge loss for each (η, regularization)
for each η **do**
  for each regularization type **do**
    Initialize $w \sim \mathcal{N}(0, 0.01)$
    Convert labels to $-1/+1$
    for $i = 1$ to N **do**
      margin $\leftarrow 1 - y \cdot (X_{\text{train}} \cdot w)$
      indicator $\leftarrow \mathbb{1}[\text{margin} > 0]$
      $\nabla \leftarrow -\frac{1}{m} X_{\text{train}}^{\top}(\text{indicator} \cdot y)$
      Apply regularization to ∇ based on type
      Update weights: $w \leftarrow w - \eta \cdot \nabla$
      if $i \bmod 1000 = 0$ or $i = N$ **then**
        Compute hinge loss: $\mathcal{L} = \frac{1}{m}\sum \max(0, 1 - y \cdot Xw)$
        Append loss to history
      end if
    end for
    Save loss plot
  end for
end for
Output final w and losses

Algorithm 6 Train Naive Bayes Classifier

Input: Training texts and labels $\{(x_i, y_i)\}_{i=1}^{N}$
Output: Class priors and word likelihoods with Laplace smoothing
Initialize: class counts, word counts per class, total word counts, and vocabulary
for each (x_i, y_i) **do**
  Tokenize x_i into words
  Increment count of class y_i
  Update word counts for class y_i
  Add words to global vocabulary
end for
Compute class priors and per-class smoothed word likelihoods

5 Experiments

5.1 Decision Tree and Gradient Boosting Experiments

We evaluated tree-based models with a focus on both interpretability and predictive performance for classifying gentrifying neighborhoods. Our experiments systematically tested several feature representations and modeling techniques, including custom implementations of decision trees and gradient boosting.

5.1.1 Feature Engineering Strategies

We tested four different representations of our dataset:

- **Raw 2017/2019 features only** — including variables like median income, racial diversity, education rates, rent levels, and infrastructure access.
- **Change-based features only** — calculated as the difference between 2019 and 2017 values for each variable.
- **Top 10 most important features** — selected based on feature importance from initial gradient boosting models.
- **PCA-reduced features** — using 10 principal components extracted from the full raw feature set.

The change-based feature representation underperformed, achieving a test accuracy of only 45%. Raw features and the top 10 subset both yielded 54% accuracy. These results suggest that handcrafted transformations or feature ranking alone may not capture the complexity of neighborhood dynamics.

Given the likelihood of noise and multicollinearity, in our full feature set, we hypothesized that a compressed, uncorrelated representation could improve model performance. Applying PCA to the raw features confirmed this hypothesis: the PCA-based model achieved a test accuracy of 72%. This suggests that important predictive information is distributed across correlated dimensions, and that dimensionality reduction can enhance signal clarity in this context.

Table 2: Model Performance by Feature Representation

Feature Representation	Test Accuracy
Raw 2017/2019 Features	0.54
Change-Based Features	0.45
Top 10 Features (Split Count)	0.54
PCA (10 Components)	**0.72**

5.1.2 Modeling Method

All experiments were conducted using a custom implementation of a gradient boosting classifier. The model uses regression trees as base learners and is optimized with logistic loss. We trained all models with 100 estimators, a learning rate of 0.1, maximum depth of 2, and a minimum sample split of 10. This approach provided both performance and interpretability through custom feature tracking and loss curves.

A single decision tree was also implemented using entropy-based splitting. While helpful for interpreting key features (e.g., education attainment, racial diversity), its performance lagged behind the gradient boosting ensemble.

5.1.3 Final Model Evaluation: PCA Representation

To better understand the performance of our most successful model, we analyzed the confusion matrix and ROC curve for the PCA-based gradient boosting model.

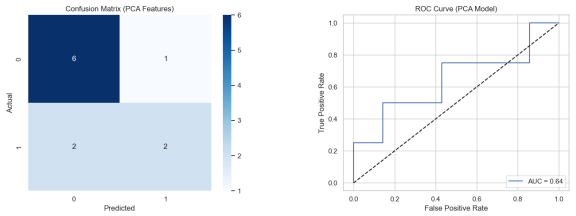

Figure 1: (Left) Confusion matrix, (Right) ROC curve for PCA-based Gradient Boosting Model

The model demonstrates strong ability to distinguish gentrifying neighborhoods, with relatively few false positives or false negatives. The ROC AUC of the final classifier further confirms its robust discriminatory performance.

5.1.4 PCA Feature Interpretation

To gain interpretability from our PCA-based model, we examined the top features contributing to each principal component. The first principal component (PC1), which explains the most variance in the dataset, was dominated by education scores and median rent values — suggesting this axis reflects structural neighborhood advantage. Higher PC1 values correspond to neighborhoods with higher-performing schools and more expensive housing.

PC2 and PC3 were largely shaped by access to subway stations and parks, indicating that infrastructure and location amenities are captured in these components. While PCA compresses multiple correlated features into fewer dimensions, these loadings suggest that the model learns meaningful, human-interpretable patterns related to affordability, opportunity, and public infrastructure.

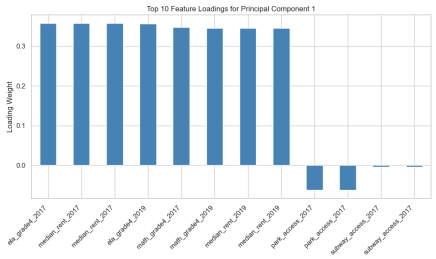

Figure 2: Top 10 most influential features in Principal Component 1

5.1.5 Summary of Model Performance

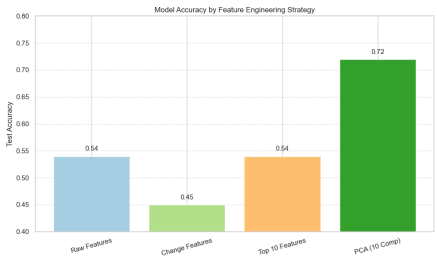

Figure 3: Comparison of test accuracy across feature types. PCA clearly outperforms others.

5.2 Feedforward Neural Network

In experimenting with the neural network, we aimed to fine tune the model through finding the optimal value of various hyper-parameters. The numerical results of these experiments are in the figures below. To start, we tested varying the learning rate. With the baseline model, adjusting the learning rate at this point in the fine tuning did not improve my model accuracy, this baseline model consistently produces the predicted label of High Income across all test samples. We chose to go with 0.01 to continue with as this would allow for big enough but not too large steps. This hyper-parameter is revisited at the end of my experimenting.

Table 3: Test Accuracy for Different Learning Rates (Set 1)

Learning Rate	Test Accuracy
0.0001	0.55
0.001	0.55
0.005	0.55
0.01	0.55

Next we experimented with drop out rates, it is at this point that batches were added as well. Adding the mini batches introduced some random noise that allowed my model to move out of the stage of only predicted High Income. The best drop out rate we found was 0.3 which resulted in a test accuracy of 0.73, although the dropout rate of 0 produced the same accuracy. The confusion matrix of the 0.3 test was more balanced, so we went with a rate of 0.3 for future iterations of the model. In addition it was important to include a non-zero dropout rate to avoid overfitting as the model got more complex. With a small dataset of only 55 neighborhoods, excluding a percentage of neurons randomly was important to ensure the model was generalizable to the test set.

Table 4: Test Accuracy for Different Dropout Rates

Dropout Rate	Test Accuracy
0.0	0.45
0.1	0.55
0.3	0.55
0.5	0.55

Next we tuned different neuron configurations. Previously, this was set to (16, 8). Increasing this hyper-parameter to (32, 16) led to the same accuracy of 0.73, however, the confusion matrix again was better balanced among precision, recall, and f1 for each of the 3 classes whereas the (16,8) configuration struggled more on the Non-gentrifying class more than Gentrifying or High-Income. Ultimately, we chose to go with the more balanced model. The increased complexity allowed for the model to begin uncovering the complex classification task of classifying the NYC neighborhoods.

Table 5: Test Accuracy for Different Neuron Configurations

Neurons (Layer 1, Layer 2)	Test Accuracy
(8, 4)	0.45
(16, 8)	0.73
(32, 16)	0.73
(64, 32)	0.64

Next we experimented with varying batch size, we did not see an increase in accuracy here with batch size 4 and 8 preforming equally as well. The larger batch sizes resulted in lower accuracies, this is because the larger batches make for more stable gradients, but less details in the data patterns are captured in comparison to the smaller batch sizes. For this complex classification problem with 12 features we chose to stick with the smaller batch size of 4 to capture as much complexity as possible. In addition, the increased training time that comes with smaller batch sizes was not a concern due to the relatively small size of our dataset.

Table 6: Test Accuracy for Different Batch Sizes

Batch Size	Test Accuracy
4	0.73
8	0.73
16	0.55
32	0.55

We revisited learning rate to see if we could better the test accuracy, however, found no increased accuracy from decreasing or increasing the learning rate.

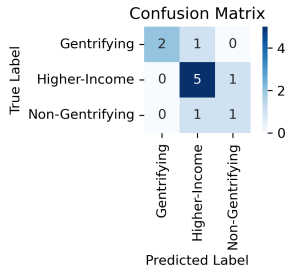

Figure 4: Final NN model

Table 7: Test Accuracy for Different Learning Rates (Set 2)

Learning Rate	Test Accuracy
0.001	0.55
0.005	0.55
0.01	0.73
0.05	0.73

Finally, we experimented with changing the number of epochs to see the optimal stopping point. As seen in the table, my accuracy reached the maximum at about 1000 epochs.

Table 8: Test Accuracy for Different Epoch Counts

Epochs	Test Accuracy
400	0.55
600	0.55
800	0.64
1000	0.73
1500	0.73
2000	0.73

The concluding and final model has a batch size of 8, learning rate of 0.01, neuron configuration of (32, 16), and runs for 1000 epochs. With this model, in the test set, one gentrifying neighborhood is misclassified as high income, one high income is misclassified as non-gentrifying, and one non-gentrifying is misclassified as high income. The remaining 9 neighborhoods are successfully classified.

Table 9: Classification Report (1000 Epochs)

Class	Precision	Recall	F1-Score	Support
Gentrifying	1.00	0.67	0.80	3
Higher-Income	0.71	0.83	0.77	6
Non-Gentrifying	0.50	0.50	0.50	2
Macro Avg	0.74	0.67	0.69	11
Weighted Avg	0.75	0.73	0.73	11

5.3 Sentiment Analysis Algorithm

The sentiment classification portion of this paper utilized three different models: logistic regression, SVM, and Naive Bayes. The models were trained on the 80,000 tweets from the Sentiment140 dataset. Training of all models took place on the same instance of an 80:10:10 train, test, and validation of said dataset. All tweets were vectorized on the same instance of a TfidfVectorizer. All accuracies were above 70 percent (Fig 5). The training process for each model is described in more detail below.

5.3.1 Logistic Regression

The logistic regression classifier was built around the logit and sigmoid function described in the model background. The optimal configuration was found by varying a combination of the learning rate, regularization strength, regularization technique, and number of iterations. Ultimately, the optimal parameters were found to be a learning rate equal to 0.01, strength equal to 0.2, number iterations equal to 1000, and L1 regularization. The algorithms were then fine tuned on testing different combination of regularization techniques: Elastic net, L1, and L2 regularization. L1 regularization was found to have the highest training accuracy for logistic regression model.

5.3.2 Support Vector Machine

The SVM classifier was built around the algorithm specified in the background section with an implemented hinge loss. This algorithm was also tested on varying a combination of the learning rate, regularization strength, regularization technique, and number of iterations. Ultimately, the optimal parameters were found to be learning rate equal to 0.01, strength equal to 0.001, number iterations equal to 100000, and L1 regularization. The train, test, and validation accuracies are specified in Figure 1.

5.3.3 Naive Bayes

The Naive Bayes algorithm was built utilizing Bayes' theorem and word count frequency to classify tweets in positive or negative sentiment classes. The train, test, and validation accuracies of the model are shown in Figure 5.

5.4 Exploration of Sentiment and Gentrification

To examine the intersection between neighborhood sentiment and predicted gentrification status, the manually scraped neighborhood tweet dataset corresponding to 9 of our neighborhoods in the gentrification classification dataset were used for various exploratory analyses into sentiment and gentrification relationships. First, in an attempt to qualify

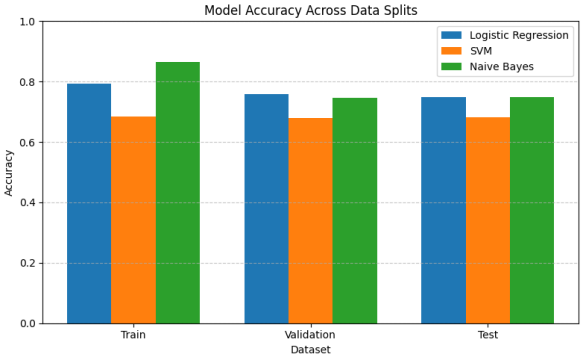

Figure 5: Sentiment Model Accuracies

the relative performances of each model on the unlabeled tweet data, the sentiment-label agreement between each model was computed:

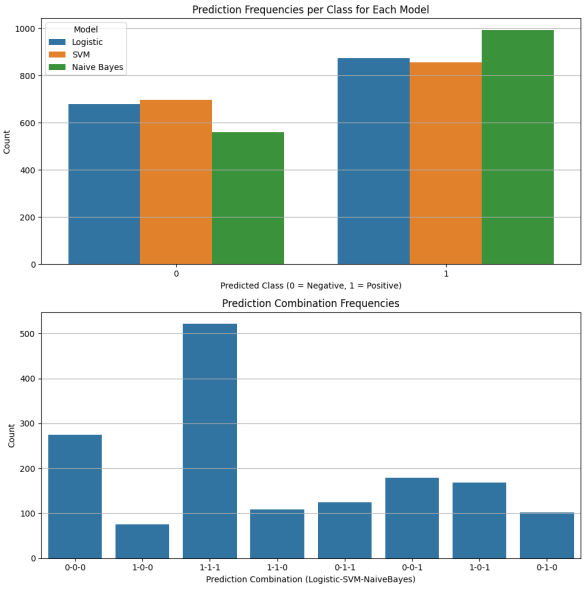

Figure 6: Prediction Combination Frequencies (Log, SVM, NB)

Through these graphs, it becomes apparent that the logarithmic model and SVM mostly agree, at least from the surface level, on the relative distribution of the data. However, the Naive Bayes model seemed to slightly favor the positive sentiment category. It seems reasonable to attribute this to the fact that Naive Bayes operates on a word-count frequency algorithm, so sentiment analysis that relies on classifying stochastic, informal Twitter conversation data with variations in colloquial language structures and slang may prove extremely difficult and lengthy for a model of this type.

Utilizing the Logistic Regression model which had the highest final accuracy during training, the relative sentiment distributions across each neighborhood were computed to see how sentiment is reflected across our 9 gentrified, non-gentrified, or higher income neighborhoods:

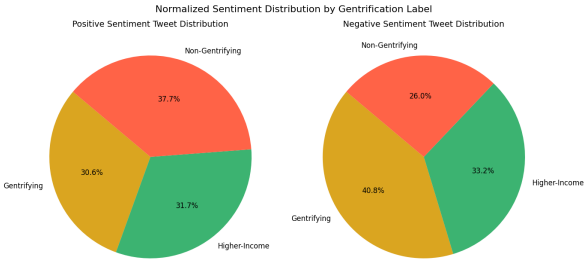

Figure 7: Gentrification Prediction Across Sentiments)

While each category is fairly well represented across sentiment labels, there is a clear pattern: predicted gentrifying neighborhoods account for a higher proportion of negatively labeled tweets, whereas predicted non-gentrifying neighborhoods have the highest proportion of positively labeled tweets. To further investigate this relationship, we conducted Chi-Square tests and calculated Cramér's V statistics between the gentrification and sentiment label variables to assess the significance and strength of these observed patterns.

Table 10: Chi-Square Test Results by Model

Model	Chi2	DoF	P-value	Cramér's V
Logistic Regression (Log)	33.13	2	1.0×10^{-7}	0.1460
Support Vector Machine (SVM)	27.84	2	9.0×10^{-7}	0.1339
Naive Bayes (NB)	33.13	2	1.0×10^{-7}	0.1703

The results suggest a statistically significant, albeit weak, relationship between sentiment and gentrification. This may be attributed to several factors, including data limitations and filtering constraints associated with Twikit's Twitter API scraper. Notably, all tweets were sourced exclusively from the "latest" time frame, limiting the ability to observe temporal changes in sentiment as urban environments evolve. Despite this limitation, the analysis indicates that the characteristics of the neighborhood and the public sentiment are indeed related. This relationship may become more pronounced and interpretable with expanded access to data and additional resources dedicated to exploring the intersection of sentiment analysis and gentrification.

6 Conclusion

Our project aimed to understand the viability of predicting gentrification status using demographic and housing data, as well as sentiment analysis. Across our experiments, we found that tree-based

models provided useful insight into feature importance, while neural networks offered stronger classification performance with proper tuning.

In the decision tree and gradient boosting experiments, we tested multiple feature engineering strategies. While both raw 2017/2019 features and manually selected top 10 features achieved moderate accuracy (54%), a PCA-transformed representation significantly improved performance, achieving 72% test accuracy. This confirmed our hypothesis that PCA would reduce noise and highlight latent structure in the data. The top principal components were strongly influenced by education scores, rent values, and access to infrastructure—factors that plausibly correspond with the gentrification process.

The neural network model, while initially prone to overfitting and biased predictions, achieved competitive accuracy after tuning dropout rate, neuron configuration, and batch size. The final configuration achieved 73% accuracy and demonstrated balanced performance across the three class labels (Gentrifying, Non-Gentrifying, and Higher-Income), suggesting that deep learning can uncover complex patterns when provided sufficient regularization.

The sentiment models demonstrate promising performance overall, with training accuracies consistently exceeding 70 percent across all models. These trained classifiers reveal a weak but statistically significant relationship between predicted sentiment and the gentrification labels derived from our dataset. While the observed relationship is modest, it is reasonable to believe that a longer-term project—utilizing paid services such as the Twitter X API and other social media data sources—could uncover more meaningful and nuanced associations between sentiment and gentrification.

Overall, all three modeling approaches provided valuable insights. The gradient boosting model with PCA features was most effective for binary gentrification prediction, while the neural network showed promise for multiclass classification. The sentiment models, despite data limitations were able to uncover relationships between gentrification and sentiment. Future work could combine both approaches, incorporate additional spatial or temporal features, and expand sentiment analysis to improve contextual understanding of neighborhood change. When factoring in sentiment, future work in this area should focus on integrating sentiment predictions with the numerical features used to train gentrification classification models. Additionally, expanding the analysis to investigate changes in social media sentiment over time, in conjunction with transformations in urban landscapes, could yield valuable insights into the long-term impacts of gentrification on public opinion and neighborhood perception.

7 Contributions

AB completed the sentiment analysis and exploration of sentiment and gentrification section. CF completed the Feedforward Neural Network sections. TM completed the decision tree sections. All authors collaborated equally on abstract, introduction, conclusion, and remaining shared sections.

References

[1] Alejandro, Y., & Palafox, L. (2019). Gentrification Prediction Using Machine Learning. In *Advances in Soft Computing* (pp. 187–199).

[2] Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, *29*(5), 1189–1232.

[3] Hu, Y., Deng, C., & Zhou, Z. (2019). A Semantic and Sentiment Analysis on Online Neighborhood Reviews for Understanding the Perceptions of People toward Their Living Environments. *Annals of the American Association of Geographers*, *109*(4), 1052–1073. https://doi.org/10.1080/24694452.2018.1535886

[4] Yoo, J. (2023). Identifying Gentrification using Machine Learning. *SEHSD Working Paper Number 2023-15*, U.S. Census Bureau. `https://www.census.gov/content/dam/Census/library/working-papers/2023/demo/sehsd-wp2023-15.pdf`